

Why LLM traffic breaks rate limiting

Why LLM traffic breaks rate limiting

Same request rate, very different cost

256-token prompt, 64-token reply



occupies a batch slot briefly

2048-token prompt, 512-token reply



Measured on one GPU at 160 concurrent requests, the short shape cleared 165 requests per second and the long shape cleared 27.

A request-per-second limit cannot see this difference. Physical capacity can.

What flow control does at the pool

What flow control does at the pool

WHAT IT DOES

A centralized, policy-aware queue in the Endpoint Picker, in front of the model servers. It enforces priority across traffic classes, fairness across tenants, and ordering within a tenant.

It is work-conserving. If the pool has capacity, it dispatches.

It routes late. A dispatched request lands on the endpoint that just freed up or holds its prefix cache.

WHERE ITS REACH ENDS

More capacity. Flow control decides when and in what order requests dispatch, and it cannot make the pool faster.

When demand exceeds capacity, the wait still exists. Flow control moves it into the Endpoint Picker (EPP) queue and decides where and for whom time to first token is accrued, protecting generation speed for everything already running.

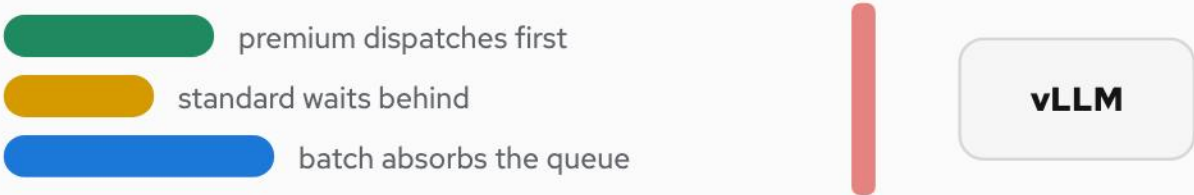
One gradient, two regimes

SATURATION UNDER THE CEILING · dispatching



Requests move straight through. Flow control is invisible.

SATURATION AT THE CEILING · dispatch halted



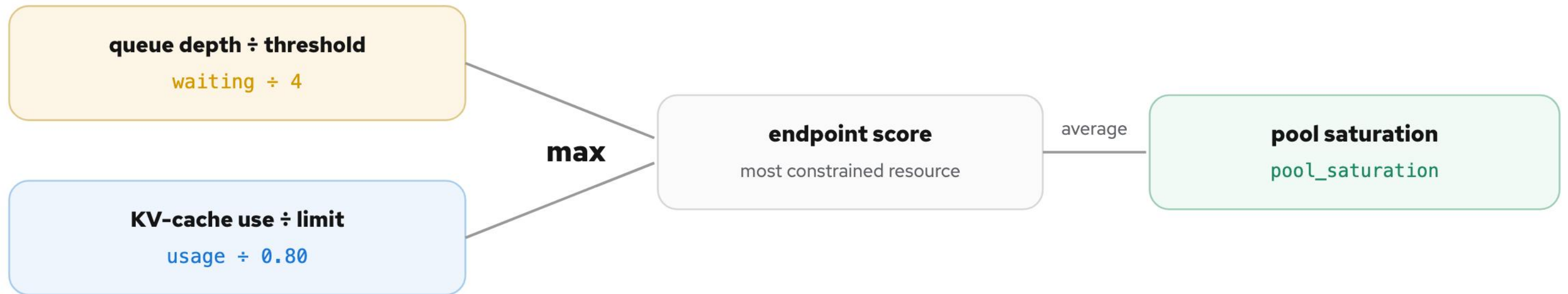
New work queues by band. The policy decides who goes next.

Saturation is a measured gradient, not a switch. Each dispatch cycle re-measures it and compares it to the ceiling.

That trade is the honest core of the system. Enabling flow control is an explicit choice to protect time per output token, the reading pace of every stream already in flight, at the price of queue time for new arrivals when the pool is full.

What flow control does at the pool

2 / 2



An endpoint whose metrics go stale is scored 100% saturated. The same detector also filters overloaded endpoints out of routing, failing open if none survive the filter.

The journey of a request

Five stops between the client and the GPU.

The journey of a request

1 / 2

1 · TAG

The trusted gateway labels traffic

x-llm-d-inference-objective → priority
x-llm-d-inference-fairness-id → tenant

2 · CLASSIFY

FlowKey = (tenant, priority)

Every FlowKey gets its own queue.
Missing objective defaults to priority 0.

Trust boundary: clients never self-assert these headers. The gateway injects them from identity.

3 · QUEUE BY POLICY

Priority bands, then fairness, then order

Band 100 · premium · dispatches first

tenant-a queue tenant-b queue

fairness cycles tenants · ordering sorts each queue

Band 0 · standard · waits behind premium

tenant-c queue

Band -10 · sheddable · bounded queue

batch queue

Per-band request and byte limits protect the gateway's memory. Past a band's limit, arrivals get HTTP 429. A queued request that outlives its time-to-live (TTL) gets HTTP 503.

4 · THE GATE

Saturation detector

Saturation is a gradient, 0 → 1+.
Under the ceiling → dispatch.
At or over → the cycle halts.
Re-measured every cycle from queue depth and KV cache, or in-flight count.

5 · LATE BINDING

The scheduler picks late

The endpoint is chosen at dispatch, where capacity or prefix cache is.

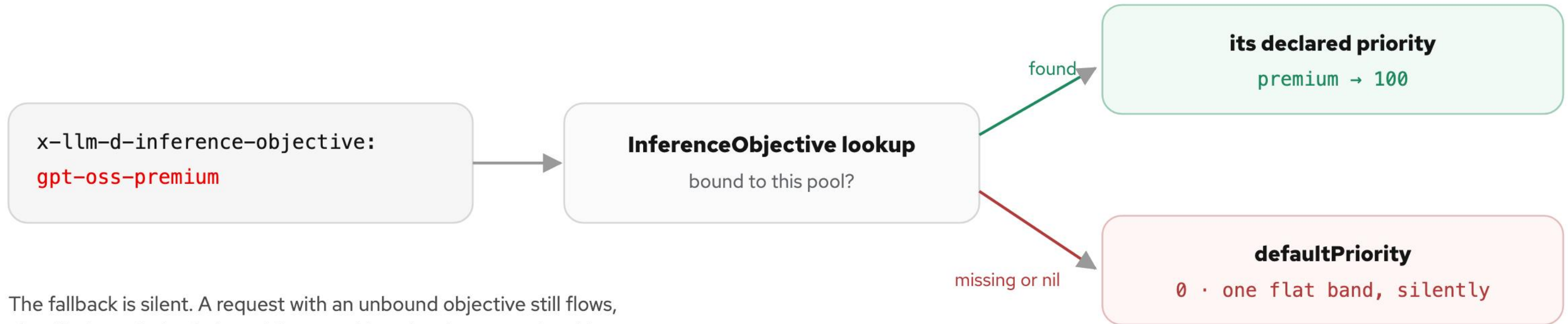
vLLM model servers

Local queues stay a small healthy buffer, deep enough that continuous batching never starves, shallow enough that the real queue stays where policy lives.

x-llm-d-inference-objective and x-llm-d-inference-fairness-id are the canonical header names. The older x-gateway-inference-objective and x-gateway-inference-fairness-id forms are accepted as deprecated aliases.

The journey of a request

2 / 2

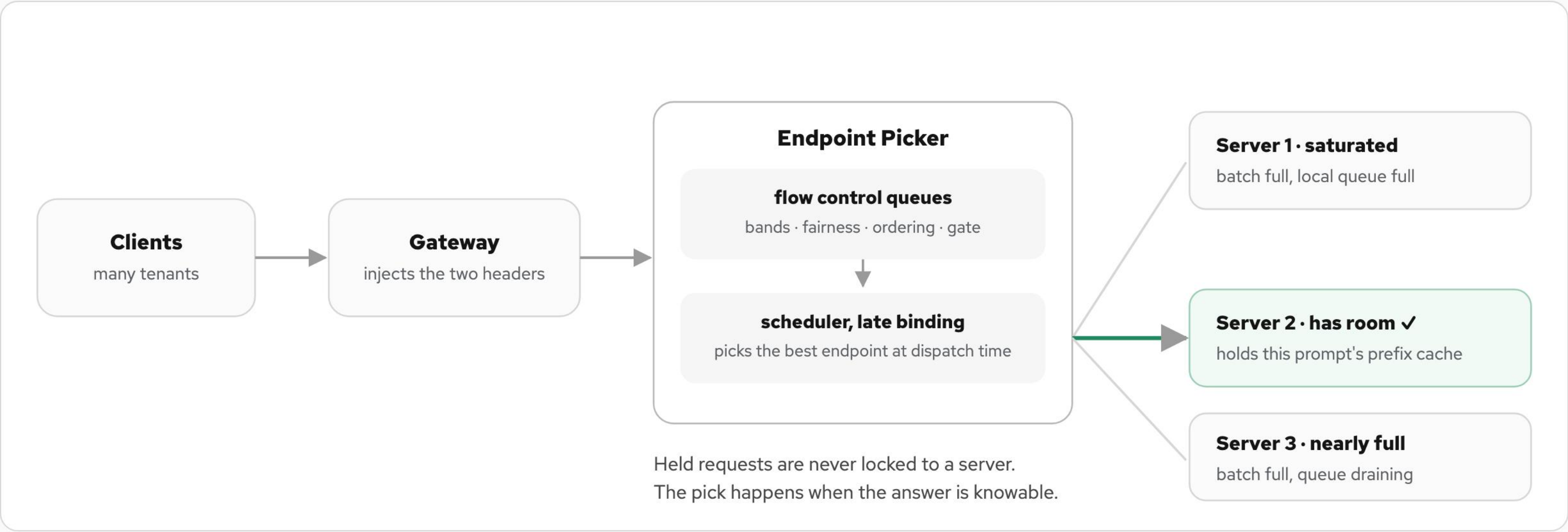


The fallback is silent. A request with an unbound objective still flows, classified at priority 0 alongside everything else that named nothing.

The architecture, one level up

The journey above happens inside one Endpoint Picker serving a pool of model servers.

The architecture, one level up

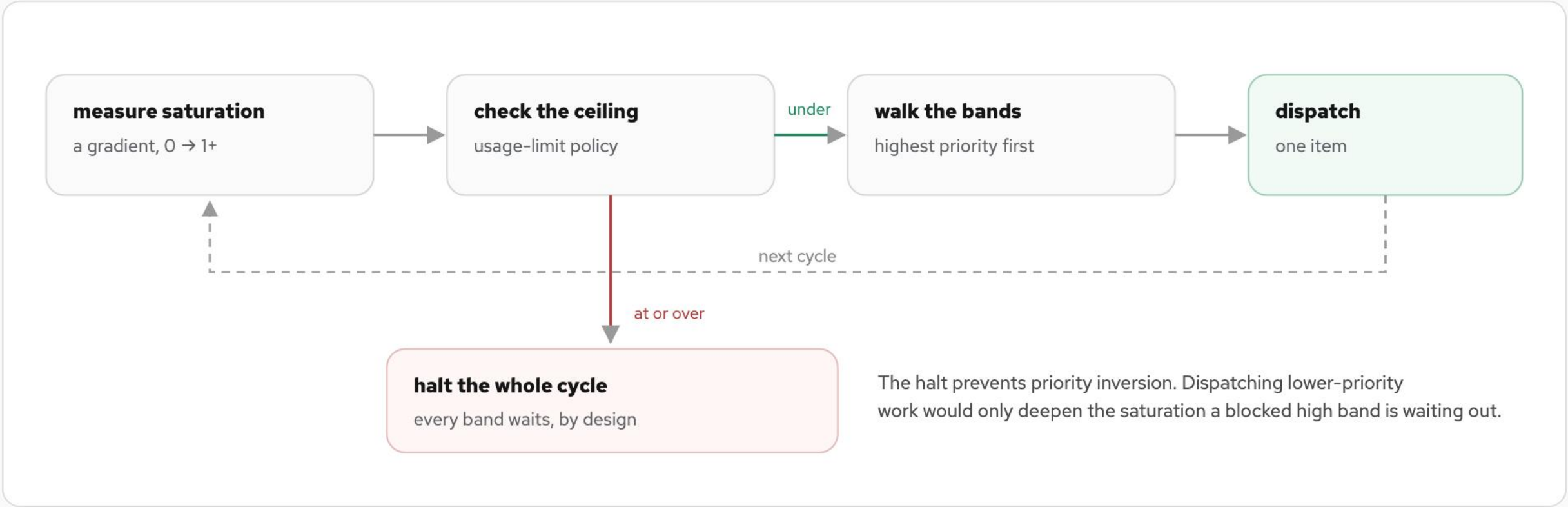


The three-tier decision

Each dispatch cycle answers three questions in strict order.

The three-tier decision

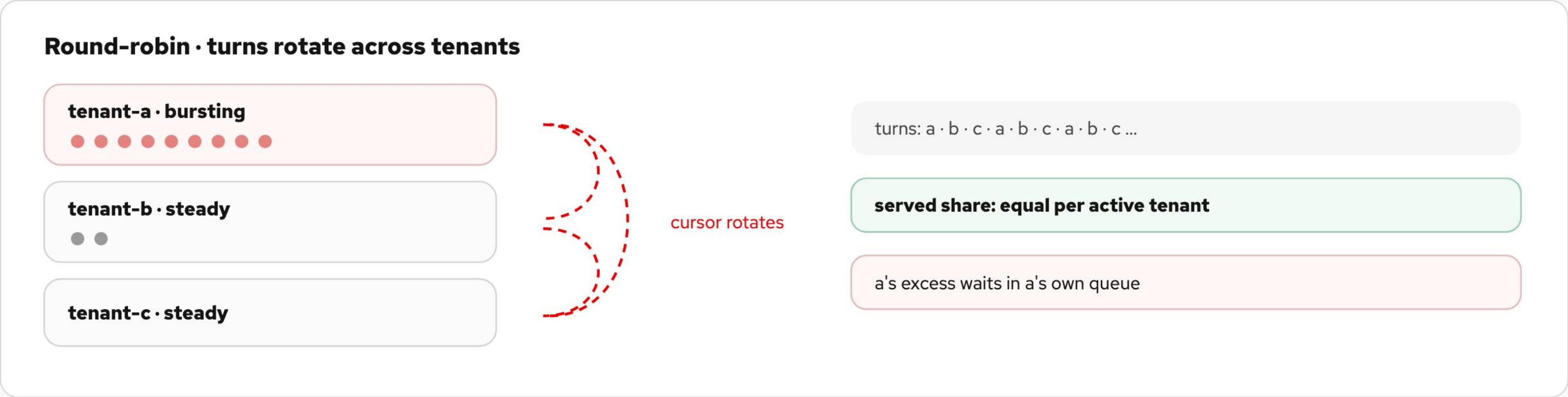
TIER	QUESTION	WHO DECIDES	OPTIONS
1 • Priority	Which band goes first?	Fixed. Highest band with work always wins	Bands come from <code>InferenceObjective</code> , negative values mark sheddable work
2 • Fairness	Which tenant inside the band?	Fairness policy plugin	Round-robin across tenants, or global strict ordering
3 • Ordering	Which request from that tenant?	Ordering policy plugin	First-come first-served, earliest deadline first, or a deadline from a service-level-objective (SLO) header



One dispatch per cycle, highest band first. In the benchmarked configuration the ceilings are uniform across bands, so that traversal order is the entire prioritization mechanism.

Fairness policies under the same burst

Round-robin's unit of fairness is the dispatch turn.



Fairness policies under the same burst

2 / 2

Global-strict · the default



one merged queue, pure arrival order

served share: proportional to volume – the burster wins

lowest overhead · no isolation

Same tenants, opposite outcome. Round-robin bounds the burster's share of dispatches; global-strict hands the band to it. To see fairness in data, measure per-tenant served share – latency alone hides it.

A third registered policy, `program-aware-fairness`, exists but is not covered here.

The three ordering policies, same queue

The same four waiting requests can be served in three different orders, depending on which signal the policy reads.

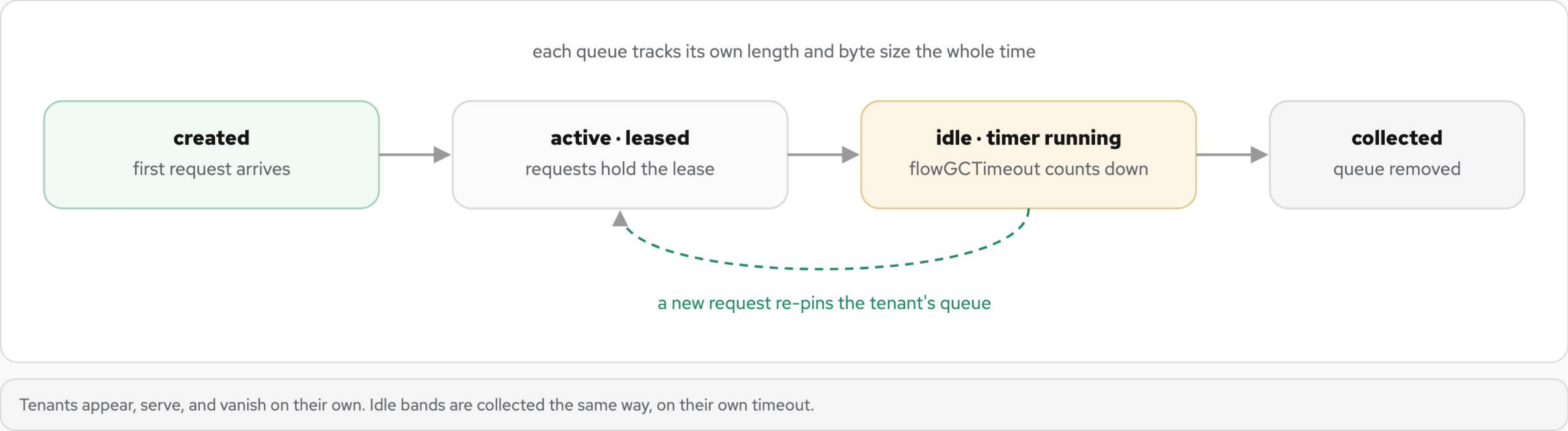
The three ordering policies, same queue



Tenants inside a band

A tenant's queue is born on its first request, held open by a lease,
and collected when idle.

Tenants inside a band



Where the waiting goes

When demand exceeds capacity, the wait has to land somewhere.

Where the waiting goes

Two waits, one budget

THE EPP QUEUE • policy decides

Time here is assigned. Priority picks the band, fairness picks the tenant, and the tenant that caused the pressure carries the most of it. Measurable per tenant and per band.

`llm_d_epp_flow_control_request_queue_duration_seconds`

THE RUNNING BATCH • physics decides

Once dispatched, a request computes inside the shared batch. A full batch produces first tokens more slowly for every request in it, whoever sent them. No scheduler reaches inside.

`vllm:num_requests_running`, `vllm:num_requests_waiting`

Tuning the saturation thresholds moves the wait between these two places. That is the dial. Let the batch fill for maximum utilization, or hold work upstream for maximum isolation, and choose per pool.

What the policy guarantees

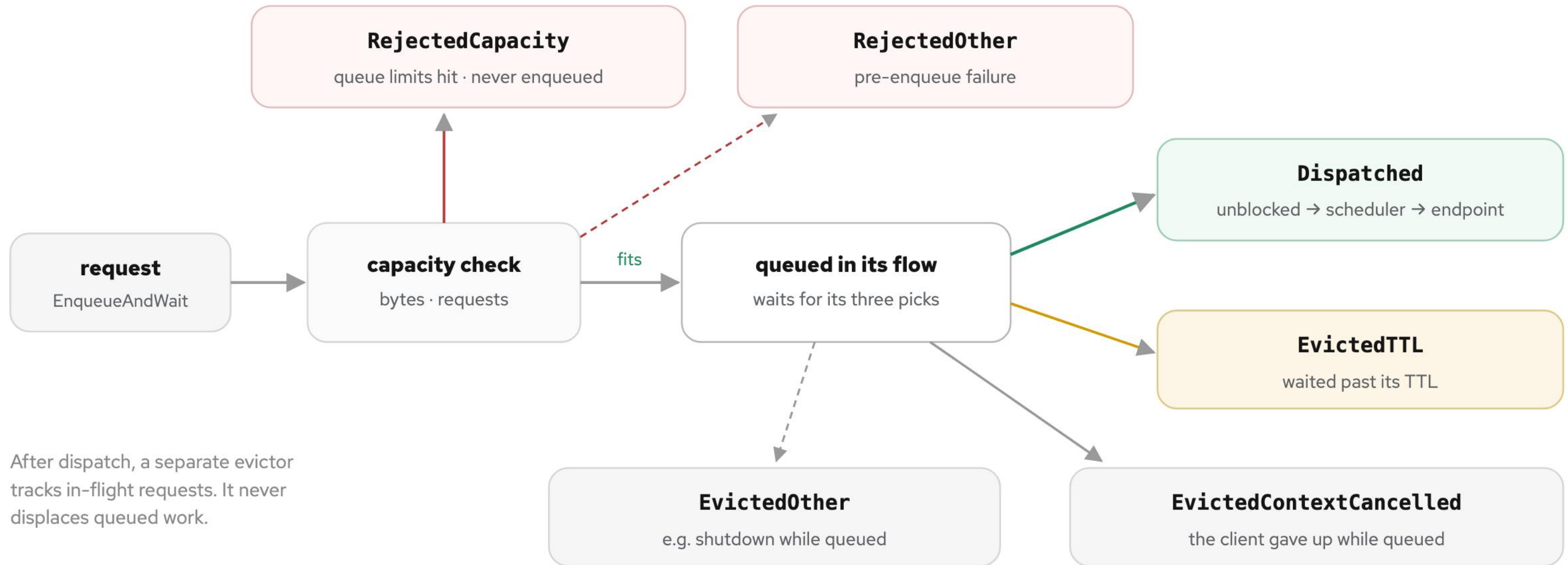
Each tier leaves a fingerprint you can measure on your own pool.

When requests are refused

Flow control fails loudly and specifically.

When requests are refused

1 / 2



When requests are refused

WHAT HAPPENED	STATUS	REASON ON THE WIRE
A band's queue limit was hit, controlled load shedding	429	rejected-saturated
A request waited past its TTL	503	rejected-ttl-expired
The client disconnected while queued	503	rejected-context-cancelled
Graceful shutdown drained the queue, retryable	503	rejected-shutting-down

Queues are in-memory.

The configuration surface

Default is what runs when you set nothing; benchmarked is what the measured configuration set.

The configuration surface

SETTING	DEFAULT	BENCHMARKED
<code>priorityBands[]</code> .priority	—	100 / 0 / -10
<code>fairnessPolicyRef</code> (per band)	<code>global-strict</code>	<code>round-robin</code>
<code>orderingPolicyRef</code> (per band)	<code>fcfs</code>	<code>fcfs</code>
<code>maxBytes</code> (per band)	1 GB each — the cap that binds	unset → 1 GB × 3 bands
<code>maxBytes</code> (global)	—	10 GiB · never binds
<code>maxRequests</code>	unlimited	unset
<code>defaultRequestTTL</code>	0 · client context only	60s
<code>expiryCleanupInterval</code>	1s	1s
<code>saturationDetector.queueDepthThreshold</code>	—	4
<code>saturationDetector.kvCacheUtilThreshold</code>	—	0.80
<code>usageLimitPolicy</code> threshold	1.0 · no gating	1.0
<code>flowGCTimeout</code> · <code>priorityBandGCTimeout</code>	idle flows and bands collected	defaults

Choosing knobs together

No knob acts alone.

Choosing knobs together

Multi-tenant isolation · benchmarked

bands	round-robin + fcfs
detector	utilization · 4 / 0.80
queue	TTL 60s · ~1 GB per band

Buys per-tenant turns in every band, floods absorbed with zero rejections.
Costs no latency help among equals; memory held while work waits.

SLO-driven interactive

bands	global-strict + slodeadline
detector	concurrency · maxConcurrency
queue	short TTL

Buys the only in-flight cap – the latency lever – plus per-request time-to-first-token targets from headers.
Costs throughput sacrificed to the cap; clients must set the SLO header.

Batch-heavy consolidation

bands	per-band maxBytes / maxRequests
detector	utilization
queue	long TTL on the batch band

Buys a bounded appetite – batch cannot eat the shared queue budget.
Costs the capped band rejects earlier, by design.

Max throughput, minimum machinery

bands	global-strict + fcfs · defaults
detector	utilization · defaults
ceiling	usage-limit 1.0 · no gating

Buys strict arrival order at the lowest overhead.
Costs a burster wins share by volume; no isolation anywhere.

Operating it

Operating it

The knobs that matter

Band priorities in `InferenceObjective` map business tiers to dispatch order. **Per-band limits** bound how much low-priority work can queue.

Request TTL caps how stale queued work can get.

The saturation detector's threshold sets the healthy buffer, deep enough that batching never starves, shallow enough that the queue stays where policy lives. It is a trigger, not a queue cap.

What to watch

`pool_saturation` is the detector's gradient; dispatch halts when it reaches the ceiling.

`queue_size` per tenant and band tells you who is waiting and how much. The same depth is a true-demand signal an autoscaler can act on.

CHECK	WHERE	PASS WHEN
Priority routing	Endpoint Picker <code>/metrics</code>	<code>queue_duration</code> with <code>priority="100"</code> is present, so premium requests really carry their band
Queues active	vLLM pod <code>/metrics</code>	<code>num_requests_running</code> sits at the cap and <code>num_requests_waiting</code> is above zero